Testing and debugging functions

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

One connected testing-and-debugging readiness record.

Explain cases, compare expected/actual, repair a defect and state coverage limits.

Keep one habit: Keep exact contract, code version, cases, first error, skipped checks and regression outcomes together.

MAIN LESSON · SECTION 1

Recall · Retrieve before notes

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

- Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.

- Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.

- Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Reading structured data

> **My remembered idea, example and correction**

Return values and local state

> **My remembered idea, example and correction**

Loops and trace tables

> **My remembered idea, example and correction**

W1 · Explain the procedure

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Separate suites: main capacity 8 expects 5,0,8 for(8,3),(8,8),(8,0); W1/W2 capacity 6 expects 4,0,6 for(6,2),(6,6),(6,0). Constant 4 passes only first W2 case; faulty addition passes only zero-used main case. Tiny contract x<3with integer 2/3/4; <=fails equality assertion, removing 3 hides it. Assertions here are normal local test checks, not input validation. Selected below_limit uses its own boundary 10.

- Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

> **My reasoning and evidence**

MAIN LESSON · SECTION 6

W2 · Predict, trace and observe

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Separate suites: main capacity 8 expects 5,0,8 for(8,3),(8,8),(8,0); W1/W2 capacity 6 expects 4,0,6 for(6,2),(6,6),(6,0). Constant 4 passes only first W2 case; faulty addition passes only zero-used main case. Tiny contract x<3with integer 2/3/4; <=fails equality assertion, removing 3 hides it. Assertions here are normal local test checks, not input validation. Selected below_limit uses its own boundary 10.

- Reference for this exercise: open this week's student lesson, "Testing and debugging functions", at "Find and repair a mistake". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- A faulty function always returns 4. Which of your three cases pass and fail?

My reasoning and evidence

MAIN LESSON · SECTION 7

W3 · Find the first error

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Separate suites: main capacity 8 expects 5,0,8 for(8,3),(8,8),(8,0); W1/W2 capacity 6 expects 4,0,6 for(6,2),(6,6),(6,0). Constant 4 passes only first W2 case; faulty addition passes only zero-used main case. Tiny contract x<3with integer 2/3/4; <=fails equality assertion, removing 3 hides it. Assertions here are normal local test checks, not input validation. Selected below_limit uses its own boundary 10.

- Reference for this exercise: open this week's student lesson, "Testing and debugging functions", at "Find and repair a mistake". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

> **My reasoning and evidence**

MAIN LESSON · SECTION 9

W5 · State an honest limit

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Separate suites: main capacity 8 expects 5,0,8 for(8,3),(8,8),(8,0); W1/W2 capacity 6 expects 4,0,6 for(6,2),(6,6),(6,0). Constant 4 passes only first W2 case; faulty addition passes only zero-used main case. Tiny contract x<3with integer 2/3/4; <=fails equality assertion, removing 3 hides it. Assertions here are normal local test checks, not input validation. Selected below_limit uses its own boundary 10.

- All three stock tests pass. Does that prove correct handling of negative counts?

> **My reasoning and evidence**

MAIN LESSON · SECTION 4

Try alone · Selected independent task

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Predict the **output**. If the function mistakenly uses x<=10, identify the first failing **assertion** and whether the final print happens. State the repair and which earlier tests must be rerun.

  Do these cases prove all **input** types are handled?

- Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Independent case: Use boundary tests to reveal a function defect

```
def below_limit(x):
    return x < 10
assert below_limit(9) is True
assert below_limit(10) is False
assert below_limit(11) is False
print("three checks passed")
```

> **My independent explanation, trace and limits**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |

MAIN LESSON · SECTION 10

Project · Keep code and evidence

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Add a labelled testing and debugging functions **evidence** sheet to Python readiness portfolio. Show how you can design revealing tests for a function.

  Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

My project change, supporting evidence and remaining limit

MAIN LESSON · SECTION 11

Feedback · Repair and return

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

Evidence	My record
Worked alone / hint / model	
First step I repaired and why	
New case checked later; date and result	
What I can now explain without notes	
One remaining question	

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Supported W2 record**

REFERENCE · SECTION 2

Reference · Supported W2 record

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use W2: Find a revealing case for Testing and debugging functions. Record the answer once; the lesson and workbook refer to the same attempt.

Given inputs	Procedure I chose	Middle steps	Result and check

> **The step I completed without help, and the hint I needed**

▼ Extra practice · Individual reasoning check

REFERENCE · SECTION 3

Extra practice · Individual reasoning check

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

> **My explanation before partner discussion**

▼ Additional practice · Earlier W4 transfer

REFERENCE · SECTION 8

Additional practice · Earlier W4 transfer

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

My reasoning and evidence

▼ **Extra practice · Pictured case**

REFERENCE · SECTION 12

Extra practice · Pictured case

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

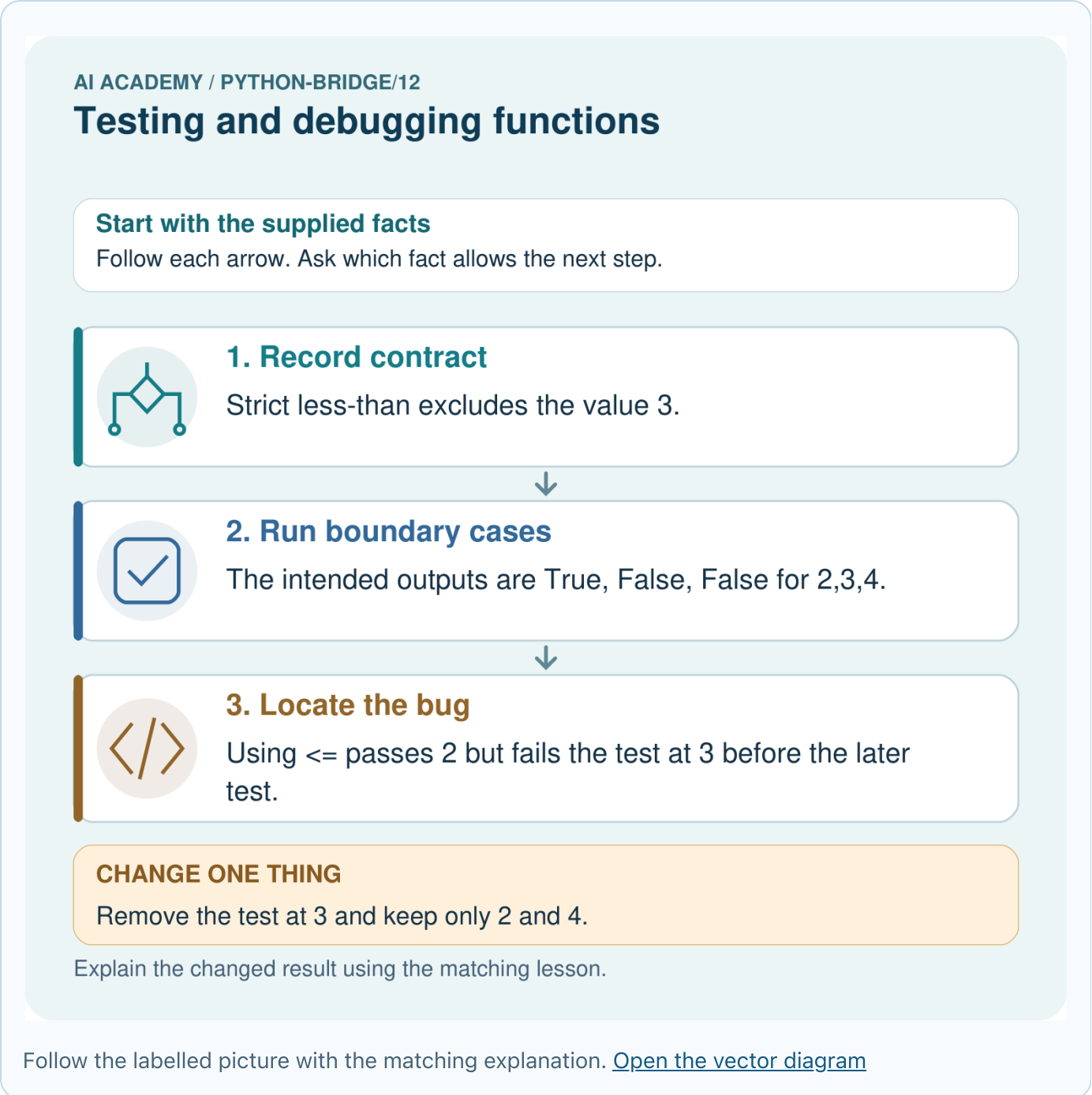
- Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False). Candidate bug: return x<=3.

- Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

- Support used: alone / hint / worked example. This record is practice **evidence**.

- Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation.

  Use actual Python 3 results to check the **prediction** after reasoning first.



Follow the labelled picture with the matching explanation. Open the vector diagram

My picture-based explanation and changed-case prediction

▼ **Reference · Project portfolio milestone**

REFERENCE · SECTION 13

Reference · Project portfolio milestone

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Project: Python readiness portfolio
- Current milestone: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.
- For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Evidence to keep	My record
This week's input and deciding step	
Prediction and actual result (or labelled paper trace)	
One change since the previous checkpoint and why	

Evidence to keep	My record
One error or limit and the next test	

> **Explain what this week adds to your project. Reuse last week's record rather than starting again.**

▼ **Extra practice · Predict, check and repair**

REFERENCE · SECTION 14

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.
- Which labelled step matches this detail? Using <= passes 2 but fails the test at 3 before the later test.
- 1. Locate the bug
- 2. Record contract

- 3. Run **boundary** cases

- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Locate the bug Using <= passes 2 but fails the test at 3 before the later test. A revealing test distinguishes a correct implementation from a plausible mistake.

Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

Changed-case question and hint

- Changed-case practice: Remove the test at 3 and keep only 2 and 4. Which conclusion is supported by the supplied facts?

- 1. The buggy <= function passes those remaining tests.

- 2. This one example guarantees correct results for every future **input**.

- 3. We can decide the result without examining the changed **input** or the procedure.

- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: The buggy <= function passes those remaining tests. Both procedures agree away from the equality **boundary**.

Repair and keep the record

Review Reading structured data (python-bridge/11). Goal: **Process** a list of records consistently. Short practice: Circle the included rows, calculate the sum and mean, and state coverage.

Repeat after adding the fourth row with count four.. Return to this week and explain what you changed.

Review Return values and local state (python-bridge/9). Goal: Use a returned value in a later calculation. Short practice: Trace the two returned values and explain why first stays seven.

What happens if the caller tries to print result directly?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

> **What I repaired, the evidence I kept and the next test**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |
> | | |

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

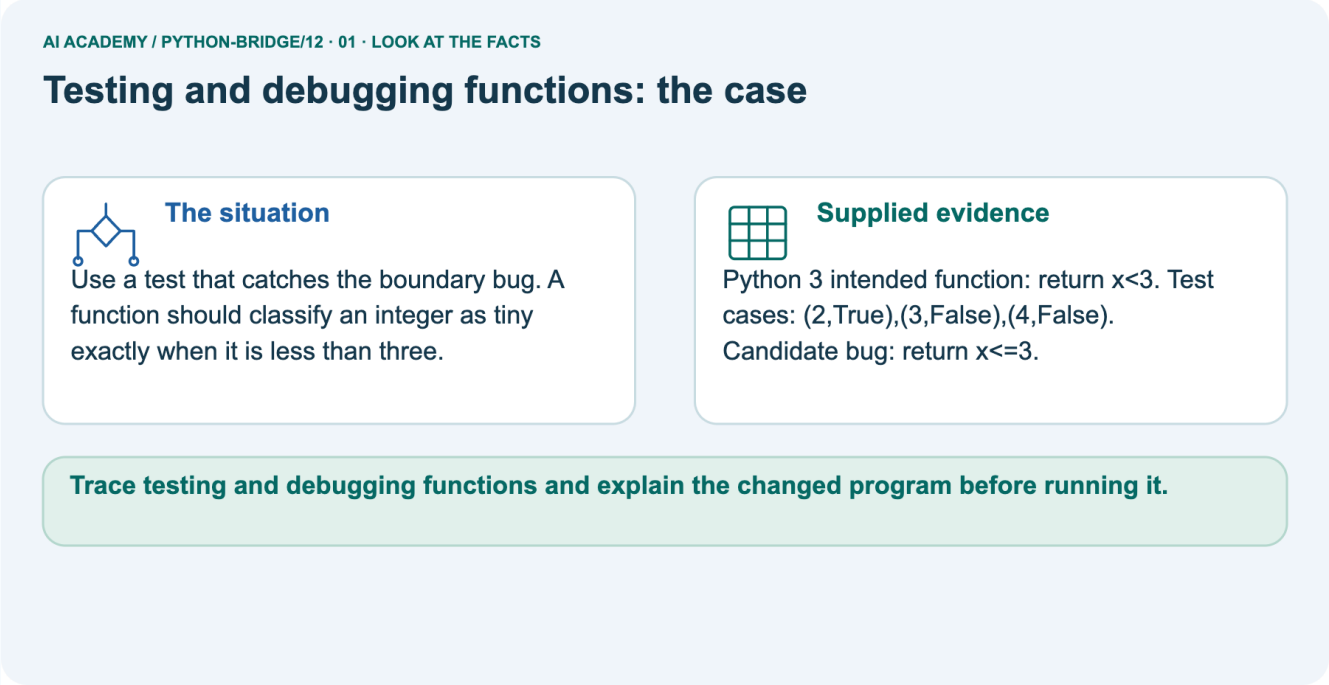
▼ **Reference · Six-picture response sheets**

REFERENCE · SECTION 15

Reference · Six-picture response sheets

> Use the picture relevant to a current gap.

01. Testing and debugging functions: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

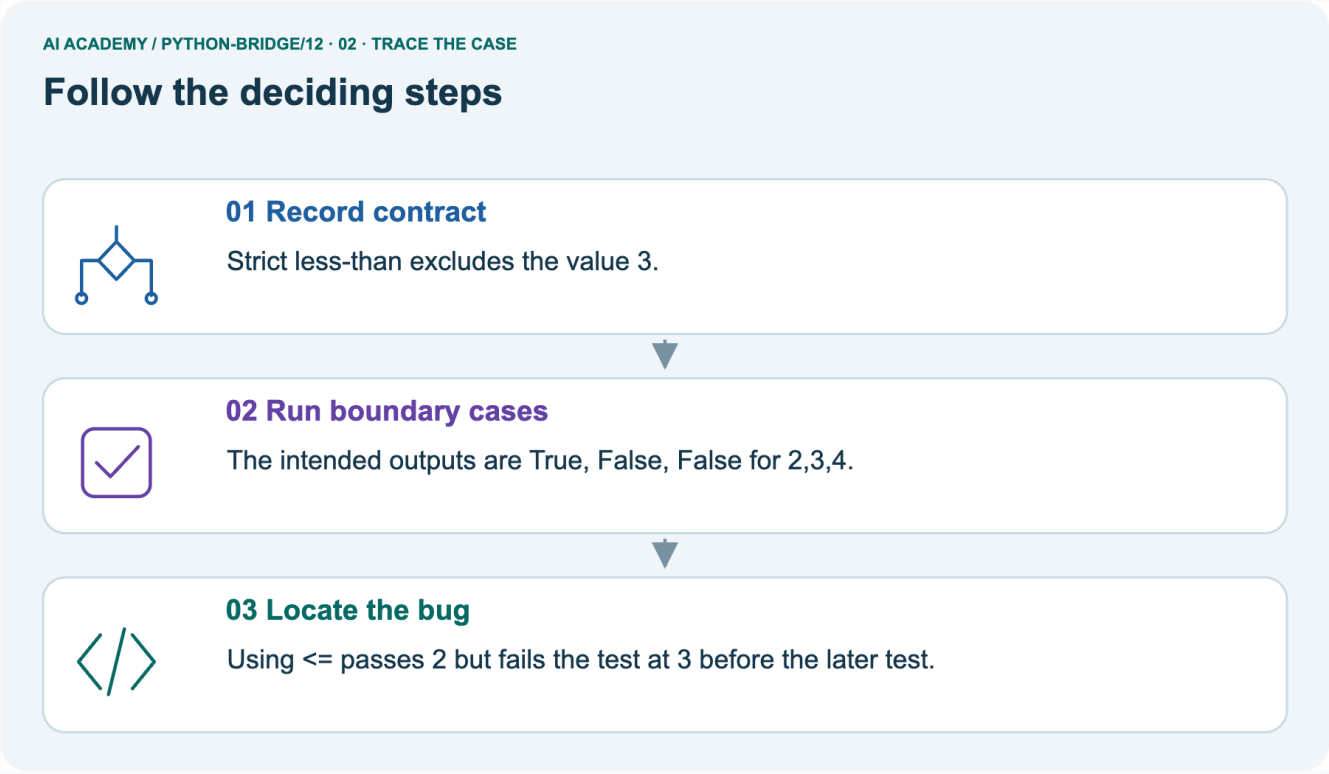
Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three. Supplied facts: Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False).

Candidate bug: return x<=3.

Point to the given fact that matters most. Explain why it matters.

My picture explanation, deciding evidence and one limit

02. Follow the deciding steps



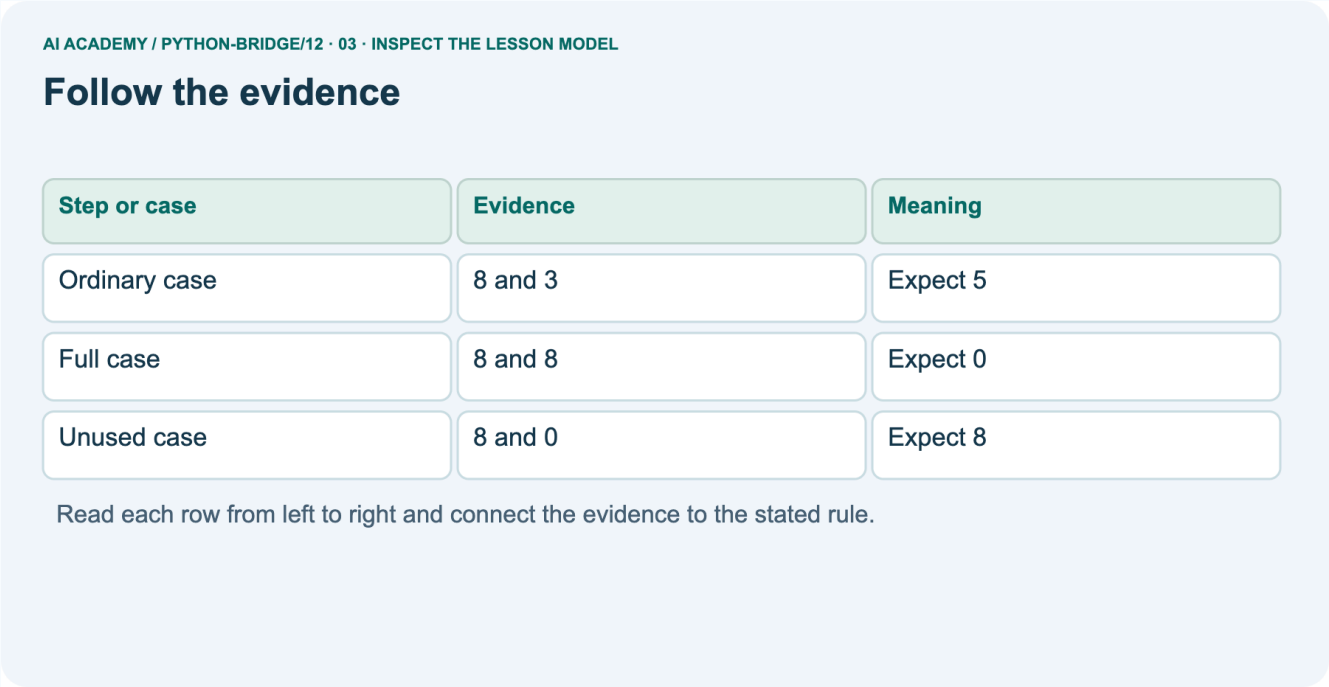
Follow the labelled picture with the matching explanation. Open the vector diagram

Record contract: Strict less-than excludes the value 3. → Run **boundary** cases: The intended **output**s are True, False, False for 2,3,4. → Locate the bug: Using <= passes 2 but fails the test at 3 before the later test.

Point to each numbered stage and explain which recorded **input** it uses.

My picture explanation, deciding evidence and one limit

03. Follow the evidence



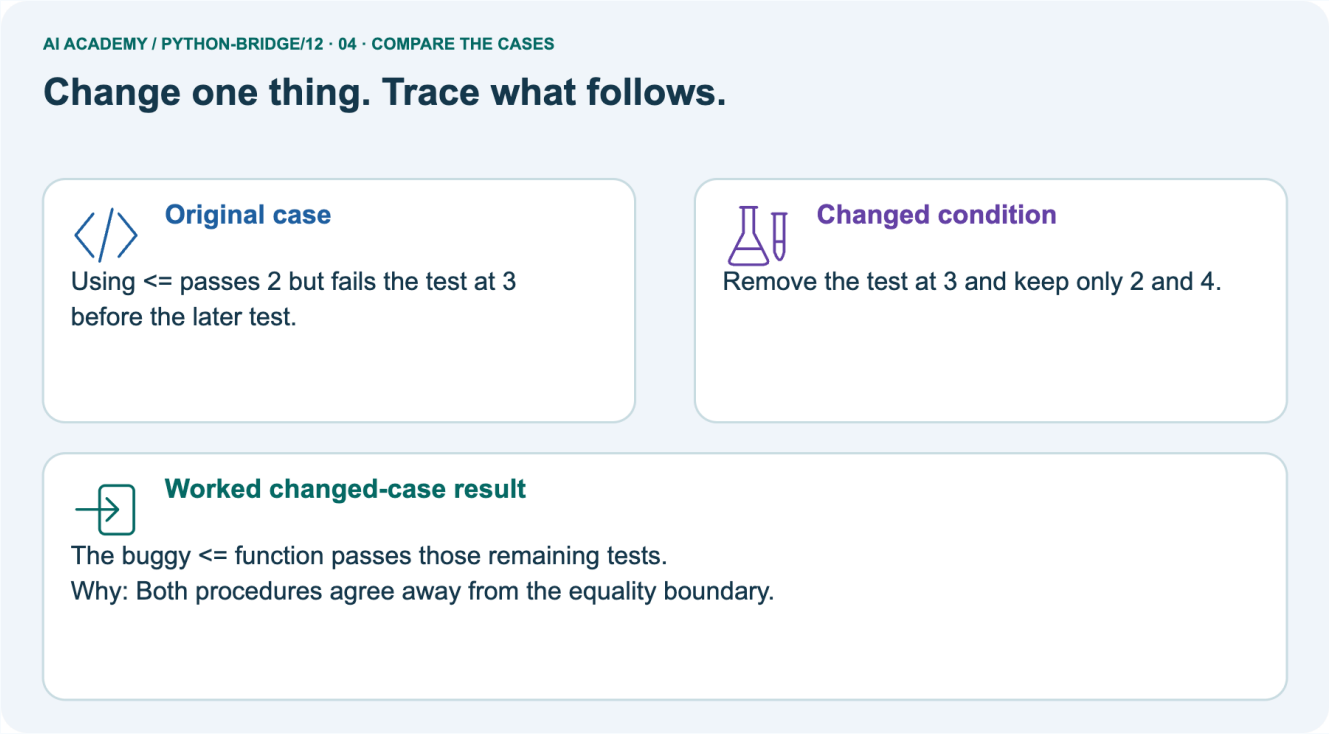
Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Ordinary case | 8 and 3 | Expect 5. Full case | 8 and 8 | Expect 0. Unused case | 8 and 0 | Expect 8

Read the labels and values. Explain the relationship the model shows.

My picture explanation, deciding evidence and one limit

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

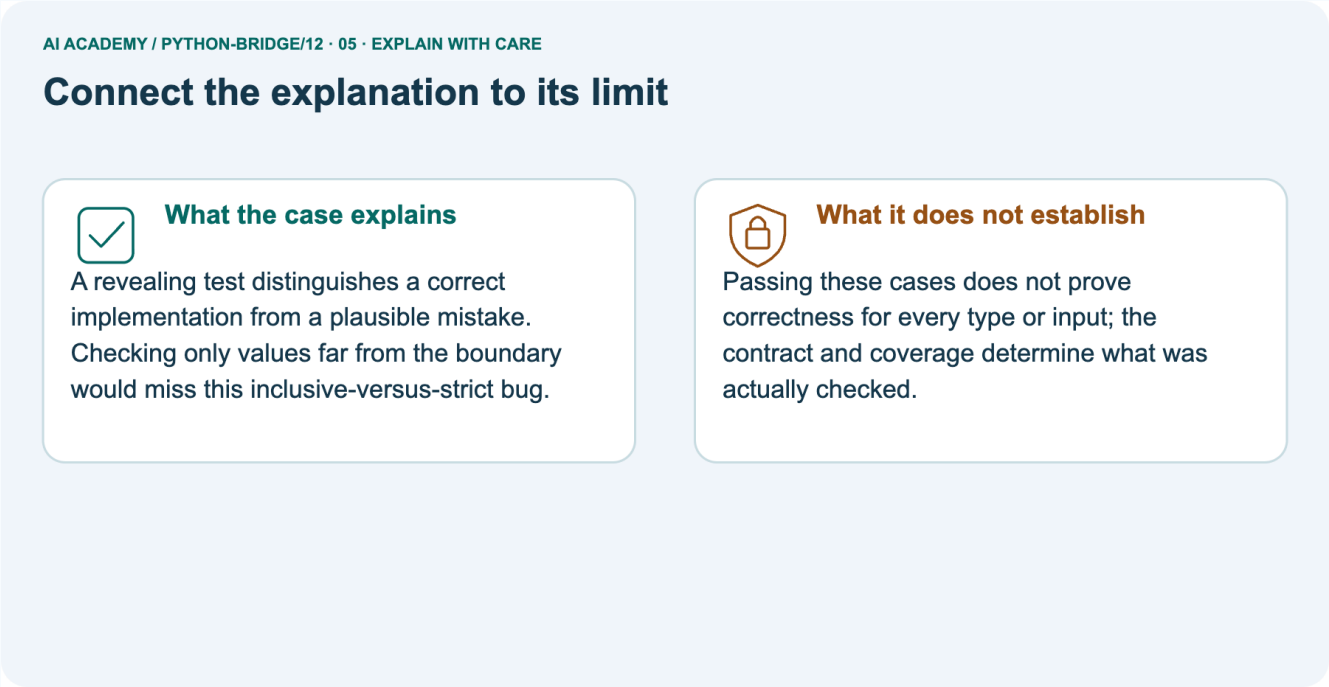
Original case: Using <= passes 2 but fails the test at 3 before the later test. Changed condition: Remove the test at 3 and keep only 2 and 4. Expected result: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

My picture explanation, deciding evidence and one limit

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

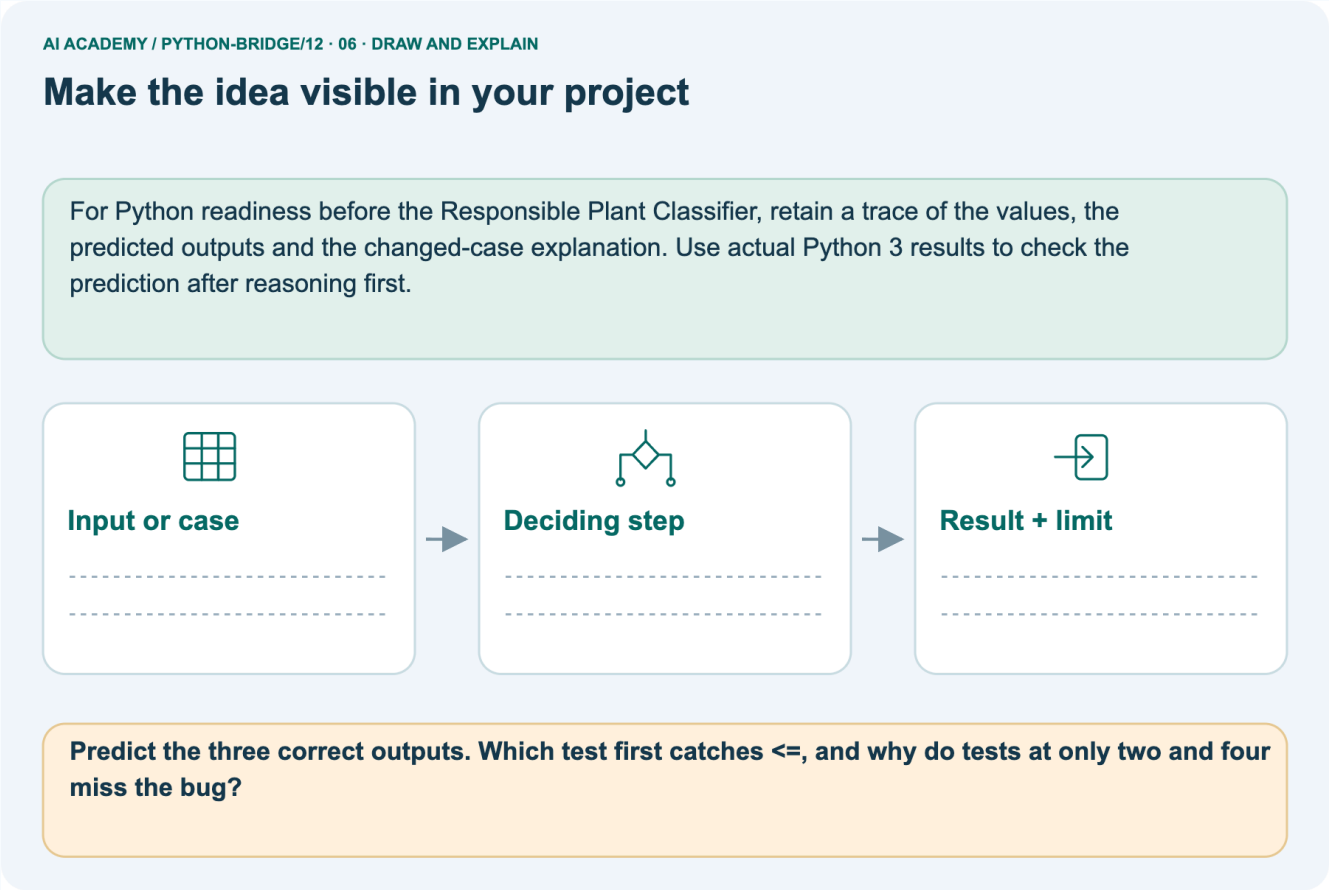
A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

Limit: Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

Explain the case, then point to a limit on the claim.

My picture explanation, deciding evidence and one limit

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

Draw your project trace and justify the deciding step using **evidence**.

My picture explanation, deciding evidence and one limit

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

01. Testing and debugging functions: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project
</details>

<details>
<summary>Optional interactive practice and completed work</summary>

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.
</details>